	UNIVERSIDAD DE SANTIAGO DE CHILE FACULTAD DE INGENIERÍA DEPARTAMENTO DE INGENIERÍA INFORMÁTICA	

TALLER DE BASE DE DATOS DIURNO 2-2026
Laboratorio 1

Ayudante: Fernando Solís	**Profesor:** Matías Calderón

Enunciado Laboratorio 1

Entrega: Semana 26 de octubre de 2026 (hasta las 12:00 hrs)

Implementación de Sistemas de Información con Arquitectura Multi-Capa, Base de Datos Avanzadas y Seguridad JWT

1. Identificación del Curso

- **Asignatura:** Taller de Bases de Datos/ Base de Datos Avanzadas
- **Nivel:** Pregrado / Ingeniería
- **Unidad Temática:** Integración de API REST, SQL Avanzado y Seguridad.

2. Descripción del Proyecto

El presente laboratorio práctico consiste en el diseño, modelado e implementación de un sistema de información integral que resuelva problemáticas de gestión de datos complejas. Se requiere el desarrollo de una arquitectura desacoplada compuesta por un Backend (API REST), un Frontend (Interfaz de Usuario) y una base de datos relacional robusta en PostgreSQL.

El foco de la evaluación reside en la capacidad del estudiante para implementar lógicas de negocio directamente en el motor de base de datos mediante objetos programables (Triggers, Stored Procedures) y optimizar el rendimiento mediante estrategias de indexación y vistas materializadas.

3. Objetivos de Aprendizaje

1. Diseñar esquemas de bases de datos relacionales normalizados.
2. Implementar lógica de servidor persistente mediante procedimientos almacenados y disparadores.
3. Desarrollar una capa de servicios (API) que garantice la integridad y seguridad de los datos.
4. Gestionar la autenticación y autorización de usuarios mediante el estándar JSON Web Tokens (JWT).
5. Documentar técnicamente procesos de despliegue y consumo de servicios.

4. Requerimientos Técnicos (Arquitectura)

- **Motor de Base de Datos:** PostgreSQL.
- **Backend:** Desarrollo de una API REST utilizando lenguajes de programación de propósito general (ej. Python, JavaScript/TypeScript, Go, Java, C#). Independiente del lenguaje no se permite el uso de ORMs. **(En consecuencia, Django queda descartado)**

- **Frontend:** Interfaz web funcional que consuma los recursos de la API (React, Vue, Angular o Vanilla JS).
- **Seguridad:** Implementación obligatoria de middleware de autenticación. Las rutas deben estar protegidas según el rol del usuario (RBAC).

5. Especificaciones de Base de Datos (Transversales)

Cada proyecto debe incluir obligatoriamente los siguientes componentes técnicos:

A. Objetos de Rendimiento y Análisis

- **Vistas Materializadas:** Implementación de al menos una vista materializada para reportes de alta complejidad computacional. Debe adjuntarse la lógica de refresco (Refrescar mediante función o evento programado).
- **Indexación:** Aplicación estratégica de índices (B-Tree, GIST, Hash, según corresponda) en un mínimo de tres columnas críticas para optimizar la latencia en lecturas.

B. Lógica de Negocio en Servidor

- **Disparadores (Triggers):** Mínimo de dos. Uno orientado a la auditoría de cambios sensibles y otro a la validación de reglas de negocio que no puedan ser controladas únicamente por restricciones de integridad referencial.
- **Procedimientos Almacenados (Stored Procedures):** Mínimo de dos. Deben encapsular transacciones atómicas que involucren modificaciones en múltiples tablas relacionadas.

A continuación, se presentan los 5 dominios. Cada grupo debe modelar la base de datos y construir los endpoints de la API para resolver las siguientes 10-12 tareas específicas.

Grupo 1: Sistema de Gestión de una Red de Clínicas Veterinarias

Contexto: Una plataforma para una red de clínicas veterinarias con varias sedes, que gestiona tutores, mascotas, veterinarios, citas médicas (en sede o a domicilio), fichas clínicas, vacunas y el inventario de farmacia de cada sede.

1. **API:** CRUD completo de todas las entidades del modelo.
2. **API/JWT:** Un tutor solo puede ver la ficha clínica de sus propias mascotas; el veterinario registra atenciones y el Administrador gestiona sedes, agenda e inventario.
3. **Procedimiento Almacenado 1:** Registrar una atención clínica completa (diagnóstico, tratamientos y medicamentos recetados), descontando los medicamentos del inventario de la farmacia de la sede en una sola transacción.
4. **Procedimiento Almacenado 2:** Aplicar un plan de vacunación a una mascota, registrando la primera dosis y generando automáticamente las citas futuras de las dosis de refuerzo.
5. **Trigger 1:** Impedir que se recete un medicamento a una mascota que tenga registrada una alergia al principio activo de dicho medicamento.
6. **Trigger 2:** Registrar en una tabla de auditoría cada vez que se modifica un diagnóstico de una ficha clínica ya cerrada, guardando el valor anterior, el nuevo, el usuario y la fecha.
7. **Vista Materializada:** "Estadísticas de Atención por Especie y Sede" que muestre la cantidad de atenciones y los diagnósticos más frecuentes en los últimos 12 meses, actualizada diariamente.
8. **Índices:** Aplicar índices en el identificador de la mascota y en la fecha de la cita para acelerar la consulta de fichas y agendas.
9. **API:** Endpoint para listar las mascotas con vacunas vencidas o próximas a vencer en los siguientes 30 días.
10. **API:** Endpoint que reciba el ID de un veterinario y una fecha, y devuelva sus bloques horarios disponibles calculados desde la base de datos.

Grupo 2: Sistema de Arriendo de Bicicletas y Scooters Compartidos

Contexto: Una aplicación para gestionar estaciones de anclaje, vehículos (bicicletas y scooters eléctricos), usuarios, billeteras de saldo, viajes y tarifas.

1. **API:** CRUD completo de todas las entidades del modelo.
2. **API/JWT:** El usuario inicia y finaliza sus propios viajes y solo puede ver su historial; el Operador gestiona la flota y las estaciones.
3. **Procedimiento Almacenado 1:** Finalizar un viaje: registrar la hora de término, calcular el costo según la tarifa y los minutos utilizados, descontar el monto de la billetera del usuario y anclar el vehículo a la estación de destino en una sola transacción.
4. **Procedimiento Almacenado 2:** Ejecutar un rebalanceo de flota: trasladar N vehículos desde una estación con exceso a otra con déficit, actualizando ambas estaciones y registrando la orden de traslado.
5. **Trigger 1:** Impedir el inicio de un viaje si el usuario tiene saldo negativo, ya posee otro viaje activo o si la batería del vehículo es inferior al 20%.
6. **Trigger 2:** Auditar cada cambio de estado de un vehículo a "En mantención" o "Dado de baja", guardando el operador responsable, la fecha y el motivo.
7. **Vista Materializada:** "Demanda por Estación y Franja Horaria" que muestre la cantidad de viajes iniciados y finalizados por estación en cada hora del día, refrescada cada hora.
8. **Índices:** Indexar el estado del vehículo, el identificador del usuario en los viajes y la fecha de inicio del viaje.
9. **API:** Endpoint para listar las estaciones con vehículos disponibles, indicando la cantidad por tipo (bicicleta o scooter) y su nivel de batería promedio.
10. **API:** Endpoint que devuelva el resumen mensual de un usuario (cantidad de viajes, minutos totales y gasto) calculado directamente en la base de datos.

Grupo 3: Sistema de Gestión de Festivales de Música

Contexto: Una plataforma para gestionar festivales, recintos, escenarios, artistas, presentaciones (line-up), tipos de entrada, venta de tickets y control de acceso.

1. **API:** CRUD completo de todas las entidades del modelo.
2. **API/JWT:** El Productor crea festivales y arma el line-up; el asistente compra entradas y solo puede ver sus propios tickets.
3. **Procedimiento Almacenado 1:** Comprar entradas: validar el cupo disponible del tipo de entrada, crear la orden, generar un ticket con código único por cada entrada y descontar el aforo en una sola transacción atómica.
4. **Procedimiento Almacenado 2:** Cancelar una presentación: marcarla como cancelada, generar notificaciones para todos los asistentes con entradas para ese día y asignarles un crédito de compensación.
5. **Trigger 1:** Impedir programar una presentación que se superponga en horario con otra en el mismo escenario, o que un mismo artista tenga dos presentaciones simultáneas.
6. **Trigger 2:** Registrar en una tabla de auditoría cada modificación del precio de un tipo de entrada, indicando precio anterior, precio nuevo, usuario y fecha.
7. **Vista Materializada:** "Ventas y Ocupación por Festival y Tipo de Entrada", con recaudación total y porcentaje de aforo vendido, refrescada diariamente al cierre de ventas.
8. **Índices:** Indexar el código único del ticket (Hash) y el ID del festival para acelerar la validación de accesos y los reportes.
9. **API:** Endpoint para obtener la grilla de presentaciones de un día del festival agrupada por escenario.
10. **API:** Endpoint de control de acceso que reciba el código de un ticket, verifique su validez y lo marque como utilizado, impidiendo su reutilización.

Grupo 4: Plataforma de Arriendo de Alojamientos Turísticos

Contexto: Un marketplace donde anfitriones publican alojamientos (departamentos, cabañas, casas) y los huéspedes los reservan, gestionando disponibilidad, tarifas por temporada, reservas, pagos y reseñas.

1. **API:** CRUD completo de todas las entidades del modelo.
2. **API/JWT:** El huésped crea y cancela sus propias reservas y solo puede reseñar estadías completadas; el Anfitrión gestiona sus propios alojamientos y tarifas; el Administrador modera publicaciones y reseñas.
3. **Procedimiento Almacenado 1:** Confirmar una reserva: calcular el total según la tarifa de temporada de cada noche, registrar el pago, bloquear las fechas en el calendario de disponibilidad y crear la reserva en una sola transacción.
4. **Procedimiento Almacenado 2:** Cancelar una reserva aplicando la política de cancelación del alojamiento (flexible, moderada o estricta): calcular el reembolso, registrar la devolución y liberar las fechas del calendario.
5. **Trigger 1:** Impedir la confirmación de una reserva si sus fechas se traslapan con otra reserva confirmada del mismo alojamiento, si la cantidad de huéspedes supera la capacidad o si no se cumple la estadía mínima.
6. **Trigger 2:** Auditar cada modificación del precio por noche o de la política de cancelación de un alojamiento, guardando el valor anterior, el nuevo, el usuario y la fecha.
7. **Vista Materializada:** Reporte de "Ocupación e Ingresos Mensuales por Ciudad y Tipo de Alojamiento", utilizado por el dashboard de administración.
8. **Índices:** Indexar el rango de fechas de la reserva (GIST), el ID del alojamiento y la ciudad.
9. **API:** Endpoint para buscar alojamientos disponibles por ciudad, rango de fechas y cantidad de huéspedes (aprovechando los índices).
10. **API:** Endpoint que devuelva la calificación promedio y el historial de reseñas de un alojamiento, calculados desde la base de datos.

Grupo 5: Plataforma de Logística y Seguimiento de Envíos

Contexto: Un sistema para gestionar centros de distribución, paquetes, vehículos de reparto, repartidores, rutas y eventos de seguimiento (tracking).

1. **API:** CRUD completo de todas las entidades del modelo.
2. **API/JWT:** El cliente crea envíos y solo puede rastrear los suyos; el Repartidor actualiza el estado de sus entregas asignadas; el Despachador ve todas las operaciones y asigna rutas.
3. **Procedimiento Almacenado 1:** Asignar una ruta de reparto: crear la ruta, asociarle un conjunto de paquetes y un vehículo, cambiar el estado de los paquetes a "En reparto" y registrar un evento de seguimiento para cada uno.
4. **Procedimiento Almacenado 2:** Registrar una entrega fallida: incrementar el contador de intentos y reprogramar la entrega; si se superan 3 intentos, marcar el paquete como "Devuelto a origen" y generar el envío de retorno.
5. **Trigger 1:** Impedir la asignación de paquetes a una ruta si el peso o volumen total supera la capacidad del vehículo asignado.
6. **Trigger 2:** Auditar toda modificación de la dirección de destino de un paquete después de haber sido despachado, guardando la dirección anterior, la nueva, el usuario y la fecha.
7. **Vista Materializada:** "Rendimiento de Entregas por Centro y Repartidor" (porcentaje de entregas a tiempo e intentos promedio), refrescada diariamente.
8. **Índices:** Indexar el código de seguimiento (tracking) y el estado del paquete.
9. **API:** Endpoint de rastreo que reciba un código de seguimiento y devuelva la línea de tiempo completa de eventos del paquete.
10. **API:** Endpoint de reporte que cruce la vista materializada para alertar sobre repartidores o centros con menos de un 85% de entregas a tiempo.

6. Entregables y Normas de Envío

Para que el proyecto sea considerado como entregado y válido para evaluación, cada grupo deberá proporcionar un repositorio de GitHub (o GitLab/Bitbucket) que contenga la siguiente estructura y elementos:

1. **Código Fuente:** Organizado en carpetas claras para backend y frontend.
2. **Script de Base de Datos:** Archivo .sql con la definición de tablas, relaciones, triggers, procedimientos, vistas materializadas e inserción de datos de prueba (seeders).
3. **README.md (Documentación Principal):** Este archivo es el corazón de la entrega. Debe incluir:
    - Arquitectura utilizada y tecnologías.
    - **Manual de Instalación y Despliegue:** Instrucciones paso a paso para ejecutar el proyecto en un entorno local (ej. configuración de variables de entorno, comandos para levantar contenedores Docker o instalar dependencias).
    - Documentación de la API (endpoints principales y ejemplos de JSON).
4. **Carpeta Presentación:** Debe incluir las diapositivas (PDF o PPTX) que se utilizarán para la defensa del proyecto.

Regla Crítica de Evaluación (Criterio de Aceptación)

Funcionabilidad del Manual de Instalación: Es responsabilidad estricta de los alumnos asegurar que el sistema pueda ser desplegado siguiendo exclusivamente las instrucciones detalladas en el archivo README.md.

Si al momento de seguir el manual de instalación el sistema no funciona, no compila o presenta errores de configuración insalvables, **el proyecto será calificado automáticamente con la nota mínima.** Se recomienda probar el proceso de instalación desde cero en una máquina limpia antes de la entrega final.

Método de evaluación: La nota final de cada laboratorio corresponderá al promedio simple entre la nota de presentación y la nota de código. Sin embargo, para que la nota de presentación sea considerada, la nota de código debe ser igual o superior a 4,0. En caso contrario, la nota final corresponderá únicamente a la nota de código, resultando en reprobación de ese laboratorio.